

기말고사 대체과제

이름	박지성
학과	컴퓨터 공학과
학년	4학년
학번	202121017
저장소 주소	https://github.com/kkuParkJiSeong/CardioCare

1. 문제 정의와 사용 목적

본 프로젝트는 심혈관 질환(Coronary Artery Disease) 유무를 조기에 스크리닝하고 의료진의 의사결정을 보조하는 기계학습 파이프라인 개발을 목표로 합니다.
해당 시스템은 환자의 수치를 바탕으로 심장질환 발생 위험도와 분류 예측 결과를 제공하여 의료진에게 정보를 전달하는 역할을 맡습니다. 최종적인 진단과 치료 방향에 대한 의학적 판단 및 결정은 오로지 의사에게 책임하에 이루어집니다.

2. EDA 핵심 결과 (그림은 2-3 개로 제한)

UCI Heart Disease 데이터 세트에서 Cleveland 부분 집합을 가지고 EDA를 한 결과 다음과 같은 특징들을 발견할 수 있었습니다. 데이터 세트 내의 정상 환자의 비율은 54%입니다. 결측치는 'ca', 'thal' 열에만 존재합니다. 'age', 'trestbps', 'chol', 'thalach', 'oldpeak' 열은 연속치이며 해당 열의 IQR 이상치는 각각 0, 9, 5, 1, 5개 존재합니다.

3. EDA 결과에 근거한 전처리 결정.

EDA 결과에 따라 전처리 결정은 결측치 처리와 이상치 처리를 중점적으로 결정했습니다. 우선 결측치는 총 6개로 매우 작은 수이나, 'ca'는 막힌 관상동맥의 개수를, 'thal'은 심장 근육에 가는 혈류의 척도를 나타내는 심장병에 매우 중요한 정보이기에 삭제하지 않았습니다. 그리고 두 열 모두 범주형 데이터를 다루었기 때문에, 평균, KNN 대치는 사용하지 못하였고, 결국 최빈값으로 대체하기 결정하였습니다.

이상치는 이상치가 있는 행이 중요한 단서를 할 수 도 있기 때문에 삭제를 하진 않고, 해당 데이터를 이상치의 최대/최소값으로 제한을 두었습니다.

4. MLflow 실험 기반 모델 비교 표 및 최종 선택 정당화

심장병 진단 시스템에서는 실재론 양성인데 시스템이 음성으로 예측하는 FN이 치명적인 결과를 낼 수 있습니다. FN을 포함하는 재현율을 모델별로 비교한 결과 랜덤 포레스트 0.81, 서포트 벡터 머신 0.91, 로지스틱 회귀 0.88로 서포트벡터 머신이 제일 현 시스템에서는 적합한 모델입니다..

```

--- Training Logistic Regression ---
Selected features for Logistic Regression: ['age', 'cp', 'chol', 'thalach', 'oldpeak', 'ca', 'thal']
Balanced Accuracy: 0.8513
Precision: 0.8485
Recall (Sensitivity): 0.8750
F1 Score: 0.8615
Confusion Matrix:
[[24  5]
 [ 4 28]]
5-Fold CV Mean Balanced Accuracy: 0.7987
5-Fold CV Mean Recall: 0.7381
5-Fold CV Mean F1: 0.7710

--- Training SVC ---
Selected features for SVC: ['age', 'cp', 'chol', 'thalach', 'oldpeak', 'ca', 'thal']
Balanced Accuracy: 0.9014
Precision: 0.9062
Recall (Sensitivity): 0.9062
F1 Score: 0.9062
Confusion Matrix:
[[26  3]
 [ 3 29]]
5-Fold CV Mean Balanced Accuracy: 0.8048
5-Fold CV Mean Recall: 0.7281
5-Fold CV Mean F1: 0.7750

--- Training Random Forest ---
Selected features for Random Forest: ['age', 'cp', 'chol', 'thalach', 'oldpeak', 'ca', 'thal']
Balanced Accuracy: 0.8718
Precision: 0.9286
Recall (Sensitivity): 0.8125
F1 Score: 0.8667
Confusion Matrix:
[[27  2]
 [ 6 26]]
5-Fold CV Mean Balanced Accuracy: 0.7896
5-Fold CV Mean Recall: 0.7199
5-Fold CV Mean F1: 0.7582

```

5. 테스트와 패키징 — 무엇을, 왜 테스트했는가

파이프라인의 안전성과 신뢰성을 위해 4가지 단위 테스트를 설계했습니다.

예측 Shape 일치 테스트: 입력 데이터 건수와 예측 출력 건수가 동일한지 검증하여 행 누락 등의 데이터 변형을 차단합니다.

예측 확률 타당성 테스트: 출력 확률이 [0, 1] 범위 내에 있고 합이 1.0이 되는지 검증하여 다중 클래스 확장성 및 통계적 일관성을 보장합니다.

임상 범위 입력값 검증 테스트: chol이 [0, 600], age가 [0, 120] 범위를 벗어날 경우 입력 단계에서 에러(ValueError)를 발생시킴으로써 배포 환경에서 비정상적인 노이즈 값이 모델로 유입되는 것을 방지합니다.

파이프라인 결정론 테스트: 동일한 입력값을 주었을 때 시드가 고정된 모델이 항상 동일한 예측 결과와 확률을 반환함을 보장하여 진단의 재현성(Reproducibility)을 유지합니다.

6. 드리프트 결과 및 재학습

[monitor.py](#) 파일을 실행시켜 드리프트를 시뮬레이션합니다. 시뮬레이션된 결과는 [drift_metrics_timeseries.png](#) 파일로 저장되어 시각화된 결과를 확인 가능합니다.

시뮬레이션을 통해 기존 세트의 균형 정확도와 드리프트 세트의 균형 정확도는 0.0172 낮아졌으며 chol 특징의 p-value가 0.0008로 기준점인 0.05보다 낮아서 드리프트를

탐지하였습니다.

주기적인 단위로 데이터 검정을 수행하여 데이터 드리프트가 감지될 경우 재학습 파이프라인을 구동합니다.

만약 시스템을 전적으로 의지하고 피드백 루프를 자동으로 진행하게 만들어놓는다면, FN이 발생하였을때, 시스템이 해당 환자가 심장병이 악화되기 전까지 FN이 누적되고 이를 바탕으로 재학습을 하게 된다면 오진 경향을 강화하는 방향으로 학습합니다. 이러한 상황은 모델의 신뢰도를 붕괴시키는 위험한 방법입니다. 이러한 현상을 막기 위해 모델이 정상으로 판단한 환자군에서도 무작위로 임의 샘플을 추출하여 정밀 검사를 실행해봐야 합니다.

7. 서빙 선택

본 시스템의 서빙 아키텍처로 **Model-as-a-Service(MaaS)** 아키텍처가 **On-Device** 아키텍처보다 나은 선택이다. **MaaS** 아키텍처가 더 나은 이유를 3가지 이유를 들어 설명 가능합니다.

로컬 디바이스의 낮은 연산 성능에 의존에 추론을 하는것 대신 중앙 서버의 대량의 연산 성능으로 기반으로 시스템을 제공한다면 안정성있는 서비스를 제공 가능할 수 있습니다. 그리고, 환자의 심장병에 관련된 데이터는 민감한 개인 의료 정보에 해당함으로 각 디바이스들에 분산 저장되는 것보다 중앙서버에서 컨트롤하는것이 보안적인 측면에선 더 안전합니다. 마지막으로 모델을 배포한 이후, 모니터링을 지속하던 중, 데이터 드리프트를 감지하여 재학습 모델을 배포해야 할 때 서버 모델의 스왑 한번으로 모든 곳에서의 실시간 업데이트가 완료됩니다.

위와 같은 자연 시간, 개인 정보 보호, 업데이트 주기 3가지 관점에서 **Model-as-a-Service** 아키텍처가 더 좋은 선택지입니다.

8. 한계

해당 시스템은 303명의 병원 데이터를 활용하므로 통계적으로 일반화된 성능을 보장하지 않는다. 특정 단일 병원의 인프라 하에서 수집되는 데이터로 해당 병원에 집중된 그룹에 특화되어 편향이 발생할 수 있는 한계점이 있습니다.

윤리적 쟁점으로는 해당 시스템이 오진을 내렸을때의 책임을 누가 지느냐에 있습니다. 특히, 심장병이 아니라고 진단했을때 실제로는 양성일 경우가 문제 발생 가능성이 높고, 이러한 일이 발생했을때, 누구에게 책임을 물을 것인지에 대해 정해놔야 합니다.

1주의 추가 개발 시간이 주어진다면 다양한 스케일링 방식과 리샘플링 방식을 도입해서 데이터를 전처리하는데 시간을 더 쏟을것 같습니다. 결국 데이터는 많은 양의 데이터에서 온다고 생각합니다.

부록 AI 도구 사용 범위

- (1) 보일러 플레이트 코드 생성. [inference.py](#), Dockerfile, [README.md](#), ci.yml 등 다양한 보일러 코드를 생성하는데 사용했습니다.
- (2) 디버깅 및 오류 해결. **mlflow ui** 명령어를 실행했을때 나오는 오류코드가 나오는 문제와 웹 페이지상에서 모델이 보이지 않는 현상을 **ai**를 활용하여 해결하였습니다.
- (3) 어떤 작업을 하였는지 **AI**를 활용하여 구조를 재점검했습니다.
- (4) 깃허브 커밋 메세지 생성에 **AI**를 활용하였습니다.